

Инструкция пользователя

Программа сравнения файлов

Инструкция пользователя

Эта инструкция показывает основной рабочий сценарий без реальных файлов, путей и конфиденциальных данных.

1. Запуск программы

Запустите приложение из PyCharm через `main.py` или соберите `exe` по инструкции из `README.md`.

На главном экране доступны два режима:

- `Сравнение` - разовая локальная сверка двух файлов;
- `Реестр объектов` - постоянное ведение объектов, подобъектов и версий файлов.

[Изображение: `images/01-main-compare.svg`]

2. Разовое сравнение файлов

- 1 Откройте вкладку `Сравнение`.
- 2 Нажмите кнопку выбора файла.
- 3 Сначала программа примет первый файл как исходный.
- 4 После выбора второго файла программа выполнит сравнение.
- 5 Справа появится список изменений с пометками `удалено`, `добавлено` или `изменено`.
- 6 При необходимости нажмите `Экспорт отчета`.

Поддерживаются текстовые документы, `config`-файлы, KLP, PDF, DOCX, Excel, изображения и бинарные файлы.

3. Работа с реестром объектов

- 1 Перейдите во вкладку `Реестр объектов`.
- 2 Нажмите `+ Объект`, чтобы создать основной объект.
- 3 Нажмите `+ Подобъект`, чтобы добавить дочерний объект.

- 4 Можно создавать сколько угодно уровней вложенности.
- 5 Выберите объект в дереве слева.
- 6 Нажмите кнопку добавления файла.

[Изображение: images/02-registry.svg]

Для удобства используйте правую кнопку мыши по объекту или файлу. Контекстное меню позволяет быстро сравнить, проверить, удалить или открыть нужное действие.

4. Логика файлов до и после

В реестре не нужно вручную выбирать ■■■■■ ■■ и ■■■■■ ■■■■■■ каждый раз.

Программа работает по цепочке:

- 1 Первая загрузка создает базовую версию.
- 2 Следующая загрузка создает новую строку сравнения.
- 3 В колонку `Файл до` попадает предыдущий файл.
- 4 В колонку `Файл после` попадает новый файл.
- 5 Для следующей сверки новый файл становится предыдущим.

[Изображение: images/03-version-chain.svg]

Так видно, что именно изменилось между соседними версиями, а не только между самым первым и самым последним файлом.

5. Мониторинг

Вкладка ■■■■■■■■■■■■ показывает состояние выбранного объекта и его подобъектов:

- сколько файлов контролируется;
- сколько файлов изменилось;
- сколько файлов без изменений;
- есть ли ошибки чтения;
- когда была последняя проверка;
- какая версия сейчас актуальна.

Цветовая маркировка помогает быстро понять состояние:

- зеленый - без изменений;

- синий - добавлено;
- оранжевый - изменено;
- красный - удалено или ошибка.

6. Версии, история и риски

Вкладка показывает цепочку загруженных файлов.

Вкладка **■■■■■■■■** показывает события по объекту: загрузка, проверка, изменение, удаление.

Вкладка **■■■■■** помогает обратить внимание на важные изменения в config-файлах: учетные записи, доступы, сетевые правила, политики и другие чувствительные участки.

7. Сборка exe

Выполните команды из корня проекта:

```

.\.venv\Scripts\python -m pip install -e "[dev]"
.\.venv\Scripts\pyinstaller packaging\file-compare-app.spec --clean

```

После сборки появится:

```
dist\■■■■■■■■■■ ■■■■■■.exe
```

Иконка  уже подключена в `packaging/file-compare-app.spec`.

8. Что не публиковать в GitHub

Не добавляйте в репозиторий:

- реальные ``.cfg``, ``.conf``, ``.ini``, ``.klp``;
- SQLite-базы ``.sqlite``, ``.sqlite3``, ``.db``;
- папки ``.venv``, ``.idea``, ``.build``, ``.dist``;
- экспортированные отчеты и архивы;
- файлы с адресами, паролями, политиками, объектами и рабочими путями.

Эти правила уже добавлены в `.gitignore`.